

Using Ren on unified memory machines

Li Xuan Tan

July 24, 2026

(WORK IN PROGRESS)

1 Hardware

RNGeo's prod server had previously been running on apollo's GPU1 until switched over to pikachu for testing. Correctness was verified (no microarchitecture funkiness on arm64 CPU) and RNGeo timings measured; JIT captures took about the same amount of time due to being bound by the CPU Python backend, whereas bandwidth-bound replays took about 5x as long.

The two hardware setups:

- apollo (1U server in rack):
 - CPU: Dual-socket AMD EPYC 9135, 16 cores per socket (Zen 5 x86-64 microarchitecture)
 - GPU: 2x RTX Pro 6000 Blackwell (each 24064 CUDA cores) connected via PCIe 5.0x16
 - DRAM (system memory): 768 GB DDR5-6400 (12 channels x 64 GB) at theoretical max bandwidth 614 GBps
 - VRAM (GPU memory): 96 GB GDDR7 per GPU at theoretical max bandwidth 1792 GBps
 - Storage: Unconfirmed - likely NVMe boot disk, with SATA mass storage mounted at \tank measured at ≈ 0.5 GBps
- pikachu (NVIDIA DGX Spark box):
 - CPU: 20-core ARM (10x Cortex-X925 + 10x Cortex-A725)
 - GPU: 6144 CUDA cores on-chip (Grace Blackwell GB10)
 - Unified memory: 128 GB LPDDR5x-8533 (16 channels x 8 GB) at theoretical max bandwidth 273 GBps
 - Storage: 4TB PCIe 4 NVMe drive, measured at ≈ 5 GBps

2 Memory bandwidth

Computations on Ren ciphertexts are often bandwidth-constrained (ref?). This makes apollo's GDDR7-based GPUs extremely attractive for their combination of high capacity and high bandwidth. In addition, apollo also has lots of relatively fast DRAM which makes caching warm users (see RGeo field guide) feasible, but evicting to disk is extremely slow on the SATA-based mass storage.

On pikachu, bandwidth-intensive operations (eg. reading and replaying a captured JIT graph) are 5-6x slower as a result of the lower bandwidth. Disk eviction is made potentially feasible by having NVMe mass storage, and is essentially required for deployment scale since pikachu cannot keep as many warm users in memory.

TLDR: Spark is slow at replays and cannot keep data in DRAM, but has relatively fast disk to make up some of that gap. CPU performance is similar, and we haven't really hit GPU performance limits yet.

3 Memory use

During first deployment of RGeo on pikachu, we noticed unified memory use of ≈ 44 GB per warm user (as opposed to previously-observed ≈ 16 GB VRAM on apollo). This was traced to **RGeo's server not realizing user contexts by default**, as a carry-over from the apollo-era code. Since apollo's disk is slow, the default way to manage multiple RGeo users is to cache their public contexts in DRAM, which apollo has a lot of (up to 32 users). Realizing each context as it arrives would pin the polys to VRAM, and at 16 GB per user we would run out of VRAM pretty fast, so no `ctx.realize()` was done.

This means the lazy polys (one set in the cached public context, and one set in the `GraphRunner` if the per-user per-day JIT plan has been saved) remain unrealized and continue to take up memory. On apollo these lived in DRAM and didn't affect the working copy in VRAM, but caused a near-tripling of memory use on pikachu's unified memory architecture. Realizing the context into "VRAM" dropped memory use to ≈ 20 GB per user, still with some overhead over apollo's DRAM/VRAM split. Unlike on apollo, pikachu's mass storage is reasonably fast and allows us to cache contexts on disk instead (plan caching is however not currently supported).

TLDR: If handling many rotating contexts (ie. apps where multiple users each have their own single-party keys), be careful of default-realize behaviour on dGPU machines because of VRAM cap, but always realize on unified machines to avoid lazy buffer bloat.

Soowon notes that on Spark-like devices where the GPU has access to the full host memory pool, `copy_to_device` can be relaxed to referencing instead of copying - TBC for a future ren update.

Aside thought: could we use apollo’s two GPUs in parallel somehow to double the available VRAM? Because the RTX Pro Blackwell series doesn’t support NVLink (NVIDIA’s proprietary high-speed interconnect between GPUs, generally supported only on datacenter-class GPUs), all inter-GPU communication must pass through the host PCIe link, which even at Gen5 x16 tops out at 128 GBps two-way total, more than an order of magnitude below VRAM bandwidth.

The runner would need to be rewritten to minimize cross-GPU communication, and even so it’d be pretty slow. Passing a ≈ 40 MB ciphertext over PCIe 5.0x16 bidirectionally would take $\frac{40\text{MB}}{64\text{GBps}} \approx 0.7$ ms at best. With compute-time-per-operation already in the low single ms range, unclear if the gains from parallelizing *within an operation* will outweigh the communication losses + needed rewrites. What we could achieve relatively easily, at least in the context of RNGeo, is to run users’ individual gameplay sessions on separate GPUs, which would double throughput and warm-user capacity (minus some overhead).

4 Crash behaviour

On apollo, GPU out-of-memory conditions (OOMs) simply trigger a `CUDAError` and terminate the offending process. Dev and prod processes are easily separable and will not affect each other.

On pikachu, seemingly due to the nature of unified memory, an aggressive memory consumer can starve the kernel of memory and cause the machine to hang entirely until power cycle. This occurred several times during RNGeo development (see RNGeo documentation), which also brought down the prod server since the entire machine was frozen. Bootstrapping experiments were particularly challenging on pikachu - large (> 50 GB) memory transients during JIT capture meant it was practically impossible to run anything while prod was running, and even with memory watchdogs/guardrails it kept OOM-crashing.

TLDR: OOM crashes are much worse on unified memory machines because the OOM affects the entire operating system. Always use memory watchdogs and place guardrails in between test phases, and be aware of large transients like JIT captures.